Sherman Tseng 33853975

Prediction of concrete compressive strength via polynomial regression methods

1. Engineering problem

1.1. Background

Concrete mixture selection studies how the concrete compressive strength (CCS) on the quantities of its constituent materials and its curing age. A computational model of this relationship supports early screening of candidate mixtures and helps engineers identify an optimal composition. Although this is a materials engineering problem in its direct application, it extends to mechanical and aerospace engineering, where the aim is likewise to predict the physical properties of a composite material from its constituents. A similar method could, for example, estimate the compressive, tensile, shear, or bending strength of an aircraft composite part from its material makeup (such as unidirectional or woven carbon fibre) and its ply orientation (such as 0°, 45°, or 90°).

1.2. Problem definition

This project aims to investigate a suitable regression model for predicting CCS, using constituent material quantities and curing age as inputs. It does not aim to derive an optimal mixture composition that maximises strength. The project will implement a set of regression models (OLS, ridge, lasso and SVR) before evaluating them based on computational expense and prediction accuracy on withheld dataset observations. The intended set of outputs are a set of saved regression models, a set of CCS predictions for each model, as well as exported performance metrics which can subsequently be used to evaluate a suitable model. Model selection does not explicitly occur within the program.

1.3. Scope

Project scope is limited to deriving a relationship between CCS and the input quantities outlined in the supplied dataset [1]. The regression models developed by the project do not account for additional material quantities not specified by the dataset, nor do they account for any other unspecified factors, such as curing regime, which may affect CCS. Model performance and accuracy is to be evaluated solely based off the supplied dataset (i.e. by deriving a train/test split); no other datasets were used for independent final evaluation.

2. Inputs, outputs and data

2.1. Inputs

The project operates using a supplied CSV file containing 1030 rows and 9 numeric columns, adapted from a publicly available .xls dataset [1]. Note that running the program requires this dataset to be downloaded and converted to .csv format. The file has a UTF-8 byte-order mark, a header row and comma-separated values. Variable units are defined by column headers contained within the dataset.

The first eight numeric columns represent input features (i.e. material quantities and curing age) whereas the final column represents the target (CCS). Once the file is loaded, the program splits the feature and target columns into separate arrays.

Variable	Type	Domain	Observed range	Units
Cement	Feature, predicts CCS	Positive real numbers (R^+)	102–540	kg/m³
Blast furnace slag	Feature, predicts CCS	Positive real numbers (R^+)	0–359.4	kg/m³
Fly ash	Feature, predicts CCS	Positive real numbers (R^+)	0–200.1	kg/m³
Water	Feature, predicts CCS	Positive real numbers (R^+)	121.8–247	kg/m³
Superplasticiser	Feature, predicts CCS	Positive real numbers (R^+)	0–32.2	kg/m³
Coarse aggregate	Feature, predicts CCS	Positive real numbers (R^+)	801–1145	kg/m³
Fine aggregate	Feature, predicts CCS	Positive real numbers (R^+)	594–992.6	kg/m³
Curing age	Feature, predicts CCS	Positive integers (Z^+)	1–365	Days
Compressive strength	Target	Positive real numbers (R^+)	2.33–82.60	MPa

Table 1: Regression features and target

2.2. Handling of invalid data

The load_validated_csv() function is used to assess validity of the input CSV file before loading, and raises ValueError if the data fails one of the checks outlined in table 2.

Check	Response
Missing file	Catches FileNotFoundError and reports the file path that could not be found.
Unreadable file	Catches permission and other file-access errors, such as supplying a directory instead of a file.
Incorrect encoding	Reads using utf-8-sig, which supports UTF-8 files with or without a byte-order mark. Reports an error if decoding fails.
Empty file	Rejects files with no content or only blank values, and files containing a header but no data.
Malformed CSV	Uses strict CSV parsing to detect quoting errors and checks that every data row has the same number of columns as the header.
Invalid headers	Rejects blank or duplicate column names and files with fewer than three columns.
Missing values	Detects empty cells, whitespace-only cells and tokens such as NA, N/A, NaN, null and None. A blank record is treated as missing data.
Nonnumeric values	Attempts to convert each cell to a floating-point number. Entries such as "abc" are reported as invalid.
Infinite values	Rejects positive and negative infinity, including values such as 1e999 that overflow to infinity during conversion.
Insufficient data	Requires at least two data rows so that a train/test split can be performed.

Table 2: Error handling checks performed by load_validated_csv()

2.3. Outputs

Each completed fitted pipeline is stored using joblib, with one file per regression type and degree (e.g. Lasso_degree_1). Saving each pipeline preserves the polynomial feature transformation, fitted scaler

and its training means and scales, as well as the trained regression model. Regression predictions, along with the true target value and associated feature values, are saved to individual CSV files (e.g. Ridge_degree_2_train.csv and Ridge_degree_2_test.csv). This allows for direct comparison of actual target values against predicted targets.

Four additional CSVs are written to results for quantitative evaluation and validation: OLS_polynomial_results.csv, Ridge_polynomial_results.csv, Lasso_polynomial_results.csv and SVR_polynomial_results.csv. Each contains degree, train mean squared error (MSE), test MSE, test R^2, fit time (s), train prediction time (s) and test prediction time (s). Printed values are rounded to three decimals, while exports retain numerical precision.

3. Computational approach

3.1. Overview

The project uses four separate regression models to generate estimates of concrete compressive strength. Each model is run up to a certain polynomial/polynomial kernel degree. Workflow is broadly sequential as outlined below:

1. Validate CSV file.
2. Select features and target.
3. Split observations into train and test sets.
4. Construct regression pipeline for each polynomial degree.
5. Fit on training data, then save and export each model and its train/test predictions.
6. Calculate prediction accuracy (MSE, R^2).
7. Save and export model performance.

Steps 4-7 are repeated for each model type (OLS/ridge/lasso/SVR).

3.2. Regression models

Method	Degrees	Preprocessing and settings
OLS	1–4	PolynomialFeatures; include_bias=False; LinearRegression defaults
Ridge	1–6	PolynomialFeatures → StandardScaler → Ridge; alpha=0.1
Lasso	1–7	PolynomialFeatures → StandardScaler → Lasso; alpha=0.1; max_iter=100000; tol=1e−5
SVR	1–4	StandardScaler → SVR; kernel=poly; C=1; epsilon=0.1; gamma=scale; coef0=1

Table 3: Summary of regression methods

3.2.1. Polynomial regression models

Polynomial regression is run using PolynomialFeatures(), which constructs powers and cross-products up to total degree d. For eight feature inputs, the number of terms becomes:

$$q = \frac{(8+d)!}{(8!\,d!)}$$

Degree 2 therefore includes squares and pairwise interactions, allowing the strength response to curve and depend on combinations of ingredients. All linear regressors fit an intercept, so include_bias=False avoids adding a redundant constant feature.

Ordinary least squares (OLS) regression minimises the sum of squared residuals. It can become sensitive to correlated high-order terms, and tends to overfit at a comparatively small degree resulting

in a significant increase in test error. OLS regression was accepted as a credible baseline to compare alternative regression models against.

Ridge and Lasso regression impose an L2 and L1 regularisation penalty respectively to linear regression models; this should prevent overfitting and allow for more accurate models given a higher polynomial degree. For both regression types, StandardScaler() was used to standardise the 8 input features, such that they have a mean of 0 and standard deviation of 1. Note that this assumes the input features follow an approximate normal/Gaussian distribution. StandardScaler was implemented to account for the difference in numerical ranges between the single time feature and 7 material quantity features, which should also improve the numerical conditioning/stability of the 2 regression models. Hence, Ridge and Lasso were selected as alternatives in order to evaluate the effects of regularisation and feature standardisation on prediction accuracy compared to the baseline.

3.2.2. Support vector regression

OLS, Ridge and Lasso polynomial regression explicitly create polynomial features and fit their coefficients by minimising squared errors, with Ridge and Lasso adding penalties to control model complexity. Comparatively, polynomial-kernel SVR represents nonlinear relationships through kernel evaluations, without explicitly generating those features, and balances model complexity against errors outside an ε-wide tolerance band. SVR was thus considered a credible alternative as it captures nonlinear behaviour while controlling overfitting, noting that the input data is assumed to be distinctly non-linear [1].

4. Validation

4.1. Method

Validation is performed via the use of a holdout train/test split, as outlined in section 2. The data is first split using train_test_split() into 824 training samples and 206 test samples, constituting an approximate 80:20 train-test split. The split is fixed (via setting random_state = 42) to ensure reproducibility across successive runs. Each model is then fitting using the training data. For ridge and lasso regression, the scaler also learns its mean and standard deviation exclusively from training data. For model evaluation and validation, each fitted model receives input features from the test data split, and uses them to predict CCS. This is then compared to actual test CCS values which were not used to fit the model.

4.2. Evaluation metrics and influence on model selection

Test mean squared error (MSE) and R^2 were used to evaluate model accuracy. Note that MSE has units MPA^2 since the target (CCS) has units MPa. A given model is more desirable if it returns a smaller MSE and larger R^2.

For each regression type, the validation error tends to reach a minimum for a certain degree, but then increases substantially for higher degrees as the model begins to overfit and capture noise. Validation holds significant, though not complete influence over model evaluation and selection. Models that significantly underfit or overfit the data were not considered acceptable. However, a model with slightly larger test error but significantly lower train error compared to another model may still be considered favourable. A model with slightly poorer accuracy but significantly lower computational expense may similarly be considered favourable.

4.3. Limitations

A notable limitation of this validation method is that the same withheld dataset is used to compare models and select the minimum test MSE and R^2. This ensures the comparison remains fair and reproducible; however, it does not show whether the results would remain consistent with a different split; some models may perform particularly well on the chosen split.

5. Model evaluation

Tables 4-7 outline each regression type, the selected polynomial degrees and performance metrics including train and test error, test R^2, fit time and prediction time. Degree 3 regression was considered an appropriate baseline for evaluation; this offers the best train and test performance within the set of OLS regression models.

Degree	Train MSE	Test MSE	Test R2	Fit Time (s)	Train Predict Time (s)	Test Predict Time (s)
1	110.645	95.971	0.628	0.0016	0.0004	0.0002
2	53.102	55.582	0.784	0.0026	0.0006	0.0003
3	19.584	40.305	0.844	0.0132	0.0013	0.0005
4	5.058	691.558	-1.684	0.0582	0.0021	0.0007

Table 4: OLS regression performance

Degree	Train MSE	Test MSE	Test R2	Fit Time (s)	Train Predict Time (s)	Test Predict Time (s)
1	111.296	96.411	0.626	0.0015	0.0005	0.0004
2	62.332	55.619	0.784	0.0302	0.0013	0.0004
3	49.397	46.353	0.820	0.0622	0.0026	0.0007
4	44.044	44.398	0.828	0.8879	0.0037	0.0011
5	39.710	42.952	0.833	1.7132	0.0085	0.0022
6	37.187	41.782	0.838	8.2294	0.0264	0.0078
7	35.461	42.234	0.836	27.2366	0.0539	0.0128

Table 5: Lasso regression performance

Degree	Train MSE	Test MSE	Test R2	Fit Time (s)	Train Predict Time (s)	Test Predict Time (s)
1	110.645	95.970	0.628	0.0018	0.0007	0.0003
2	55.252	54.736	0.788	0.0024	0.0010	0.0004
3	25.515	43.804	0.830	0.0046	0.0020	0.0006
4	15.361	44.503	0.827	0.0165	0.0046	0.0010
5	11.670	46.533	0.819	0.0471	0.0119	0.0028
6	8.863	52.255	0.797	0.0836	0.0237	0.0067

Table 6: Ridge regression performance

Degree	Train MSE	Test MSE	Test R2	Fit Time (s)	Train Predict Time (s)	Test Predict Time (s)
1	117.371	104.361	0.595	0.0163	0.0093	0.0023
2	63.174	58.701	0.772	0.0219	0.0086	0.0020
3	48.201	51.165	0.801	0.0224	0.0077	0.0019
4	36.483	47.436	0.816	0.0353	0.0089	0.0022
5	27.204	43.159	0.833	0.1103	0.0093	0.0027
6	20.710	44.482	0.827	0.1677	0.0113	0.0020
7	16.961	41.079	0.841	0.9565	0.0079	0.0021
8	12.949	45.171	0.825	1.7402	0.0090	0.0032

9	9.807	188.283	0.269	3.7202	0.0085	0.0029

Table 7: SVR regression performance

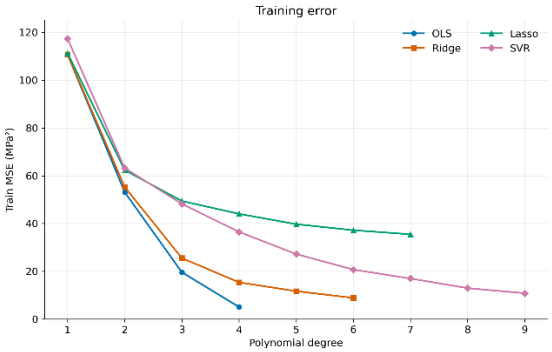


Figure 1: Train MSE comparison

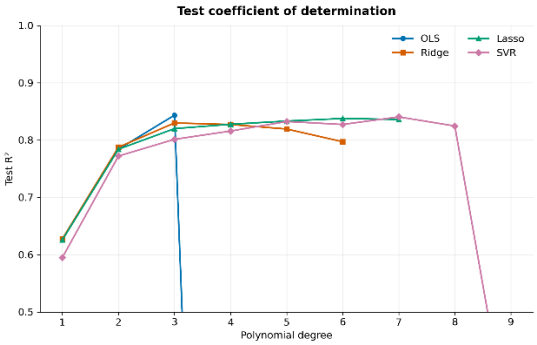


Figure 2: Test R^2 comparison

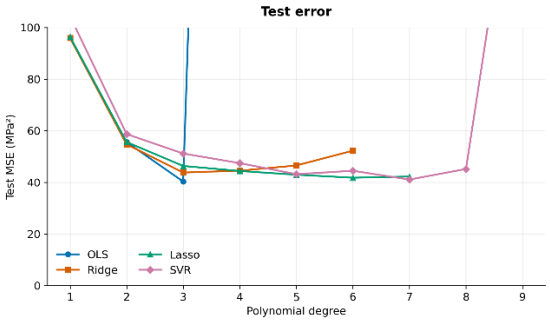


Figure 3: Test MSE comparison

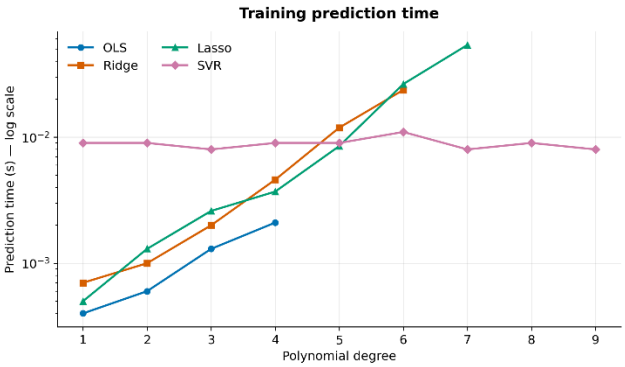


Figure 4: Prediction time comparison

Based on these results, degree-3 OLS polynomial regression is the most suitable model, achieving the lowest test MSE of 40.305 MPa² and the highest test R^2 of 0.844, explaining approximately 84.4% of the variation in test-set compressive strength. It also combines strong accuracy with low computational cost, requiring 0.0132 s to fit and 0.0005 s to predict the test set—substantially faster than the next faster model, degree-7 SVR, which achieves a test MSE of 41.079 MPa². However, its training MSE of 19.584 MPa² indicates a generalisation gap, while the sharp deterioration at degree 4 highlights the risk of increasing model complexity.

6. Optimisation and profiling

Within this project, the overall objective of the program is to output a comparison of model solutions, rather than a single solution. Hence, improvements to code performance were mainly centred around reducing the overall runtime of the code by removing runs for some less meaningful regression models. Time profilers were added to the code to examine particularly computationally expensive runs.

All regression models were initially run to degree 10. At such a high degree, however, most models exhibited significant test error due to overfitting, and were considered unusable; this also sightly increased the overall runtime of the program. The set of executed polynomial runs for Ridge, SVR and OLS regression were subsequently reduced such that only the model first exhibiting a spike in error was retained; this allows for clear inspection of where the minimum test error occurs for each regression type. For instance, OLS regression was terminated after degree 4, however, the degree 4 regression was kept to show when the model first begins to overfit.

Lasso regression was not shown to exhibit a spike in test error; rather the test error appeared to converge to around 35-36 MPa^2. Lasso regression was instead terminated when the runtime for a fitting a particular degree was found to exceed 1 minute, which occurred after degree 7.

7. Conclusion

This project aimed to use various regression methods to estimate the CCS of concrete given constituent material quantities and curing age. Four regression models (OLS, Ridge, Lasso and SVR) were run with varying polynomial and polynomial kernel degrees. Alternative models were chosen to investigate the effect of regularisation, input feature normalisation, and SVR regression on overall CCS prediction accuracy. Validation was conducted by implementing a train-test split on the input data, and withholding the test set for model evaluation.

Overall, the baseline degree-3 OLS polynomial regression was considered the most suitable model, suggesting that the effect of SVR regression, regularisation and feature normalisation is negligible for

accurately predicting CCS. Ridge, Lasso and SVR regression achieve comparable, but not markedly superior results compared to the chosen baseline regression method. Limitations

Whilst features in the program exist to check numerical validity, they do not check engineering plausibility. A more effective program would also check for negatively reported ingredient quantities, as well as fractional and negative curing ages within the input. Similarly, a more effective model implementation would also explicitly check for negative CCS predictions.

Another substantial limitation lies within the validation method. As discussed in section 4, enforcing a fixed train-test split allows for fair and reproducible model evaluation, however, this may allow a particular model to over-perform. The most effective model is also not guaranteed to accurately predict CCS when provided a completely new dataset, especially if that dataset was developed under different conditions, such as a different concrete curing regime.

A more rigorous validation method would involve reserving a test set for final evaluation, and applying k-fold cross-validation to the remaining training data. An optimal regression type and polynomial degree can then be selected using the lowest average cross-validation MSE, before being refit and evaluated on the still-unseen test data set.

8. AI use and reflection

ChatGPT was used substantially for coding assistance throughout this report. However, usage of AI was restricted to adding additional functionality, as opposed to developing and altering the core output of the program (i.e. running a set of regression models).

Baseline and alternative models used in the project were chosen independently, as were specific regression parameter such as L1 and L2-penalties. Code pertaining to running each regression model, as well as calculating validation metrics such as test error and R^2, was completed without AI usage. Likewise, code pertaining to loading the input CSV, separating feature and target data, and implementing the train-test split was conducted. Input arrays such as x_train and y_train were checked independently using print statements (later omitted in the final code) or inspection of variable definitions.

Generative AI was used to specify an appropriate set of input verification checks, as well as to explain how to develop the load_validated_csv function. This function was subsequently tested by modifying the input data to include invalid entries and checking if an appropriate error was raised by the function. Generative AI was further used to explain how to add file exportation functionality to the code for model performance and predictions, for instance how to export “Lasso_polynomial_results.csv” or “Ridge_degree_1_train.csv”. Exported files were subsequently checked to ensure they contained sensible data. For instance, the train and test MSE for a number of regression models was calculated using data stored in the associated prediction CSVs. This was subsequently checked against the MSE values printed to the code terminal, ensuring the stored predictions matched the actual predictions used for validation.

To verify changes made by AI, a separate python script (tester.py) was developed, which simply inputs the CSV, runs each regression model without a dedicated function and returns train MSE, test MSE and test R^2. The purpose of this script was to serve as a redundancy check, ensuring that changes to the main script made with guidance from AI did not result in changes to regression performance. For instance, an AI-suggested modification to the run_polynomial_regressions function resulted in a deviation in calculated performance values for SVR regression, which was subsequently reversed.

References (APA 7th)

[1] Yeh, I.-C. (2007, February 8). *UCI machine learning repository*. Archive.Ics.Uci.Edu. https://archive.ics.uci.edu/dataset/165/concrete+compressive+strength